Largest Park

AlgoExpert · Very Hard · Stacks

The problem

- Boolean matrix where false is free land and true is in use.
- Area of the largest all-free rectangle (0 if none).
- For each row build a histogram of consecutive free cells above and take the largest rectangle in it with a monotonic stack.

Intuition

- Treat each row as a histogram of free heights and apply the largest rectangle in histogram.

Brute force vs optimal

- Check every rectangle → $O(R^2C^2)$
- Histogram per row + monotonic stack → $O(R \cdot C)$

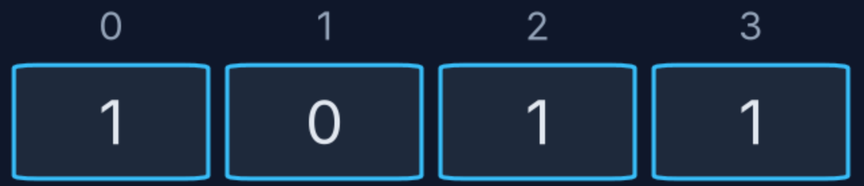
Approach

- 1. Update column heights per row
- 2. Run the monotonic stack
- 3. Track the max area

Worked example

0	1	0	0
0	0	0	0
1	0	0	1
0	0	0	0

free heights



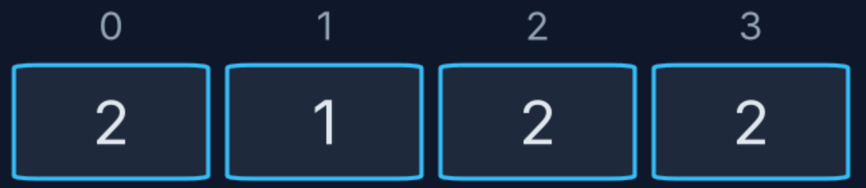

```
largest rectangle in histogram: 2
```

Row 0: histogram [1, 0, 1, 1]

Worked example

0	1	0	0
0	0	0	0
1	0	0	1
0	0	0	0

free heights




```
largest rectangle in histogram: 4
```

Row 1: histogram [2, 1, 2, 2]

Worked example

0	1	0	0
0	0	0	0
1	0	0	1
0	0	0	0

free heights




```
largest rectangle in histogram: 4
```

Row 2: histogram [0, 2, 3, 0]

Worked example

0	1	0	0
0	0	0	0
1	0	0	1
0	0	0	0

free heights




```
largest rectangle in histogram: 6
```

Row 3: histogram [1, 3, 4, 1]

Java solution (1/2)

```java
public class LargestPark {
    public static int largestPark(boolean[][] land) {
        if (land.length == 0) return 0;
        int[] heights = new int[land[0].length];
        int best = 0;
        for (boolean[] row : land) {
            for (int c = 0; c < row.length; c++) heights[c] = row[c] ? 0 : heights[c] + 1;
            best = Math.max(best, largestRectangle(heights));
        }
        return best;
    }
```

Java solution (2/2)

```
    static int largestRectangle(int[] h) {
        ArrayDeque<Integer> st = new ArrayDeque<>();
        int best = 0;
        for (int i = 0; i <= h.length; i++) {
            int cur = i == h.length ? 0 : h[i];
            while (!st.isEmpty() && h[st.peek()] >= cur) {
                int height = h[st.pop()];
                int left = st.isEmpty() ? -1 : st.peek();
                best = Math.max(best, height * (i - left - 1));
            }
            st.push(i);
        }
        return best;
    }
}
```

Complexity

- Time: O(w*h)
- Space: O(w)

Edge cases & pitfalls

- Reset height on obstacles
- Empty grid → 0

Interview tips & follow-ups

- Same as Maximal Rectangle

Recap

- Treat each row as a histogram of free heights and apply the largest rectangle in histogram.
- Complexity: O(w*h)